

JOBPILOT

AI-Powered Job Application Tracker and Career Intelligence Platform

CAPSTONE PROJECT PHASE 2

DSN4091 - Capstone Project Phase 2

Submitted by

Ishaan Verma – 23BCE11052

Harshit Arora – 23BCE10911

Udit Narayan Rout – 23BCE11013

Kartikey Tiwari – 23BCE11650

Aashish Pandey – 23BCE11066

in partial fulfillment of the requirements for the degree of
Bachelor of Technology



VIT Bhopal University
Bhopal, Madhya Pradesh

Dr. S. Poonkuntran
Executive Dean , VIT BHOPAL UNIVERSITY
2026 -2027

TABLE OF CONTENTS

Section	Page
Chapter 1: Introduction	6
1.1 Motivation	6
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Scope	6
Chapter 2: Existing Work / Literature Review	8
2.1 Information Extraction from Résumés	8
2.2 Semantic Matching and Recommendation	8
2.3 Explainable Decision Support	8
2.4 AI Assistants and Structured Outputs	8
2.5 Relevance to JobPilot	8
Chapter 3: Frontend, Backend and System Requirements	9
3.1 Technology Stack	9
3.2 Software Requirements	9
3.3 Hardware Requirements	9
Chapter 4: Methodology	10
4.1 System Design and Architecture	10
4.2 Frontend Design	10
4.3 Résumé Ingestion and Profile Processing	10
4.4 AI Integration and Validation	11
4.5 Job Catalogue and Matching	11
4.6 Application-Readiness Scoring	11
4.7 Skill Evidence and Career-Twin Logic	12
4.8 Next-Best-Action Workflow	12
4.9 Application Records, Documents and Reminders	12
4.10 API and Data Model	13
4.11 Authentication and Security Considerations	13
4.12 Feature Status: Implemented and Limitations	14
4.13 Testing Strategy and Available Evidence	14
4.14 Results and Discussion	14
4.15 Individual Contributions	15
Chapter 5: Conclusion	16
5.1 Conclusion	16
5.2 Limitations	16
5.3 Future Enhancements	17
5.4 Project Screenshots	17
5.5 Final Outcome	20
References and Publications	21

LIST OF FIGURES

Figure	Title	Page
Figure 4.1	JobPilot logical system architecture	10
Figure 4.2	Résumé ingestion and profile-processing pipeline	11
Figure 4.3	Six-factor application-readiness workflow	12
Figure 4.4	Application-data lifecycle for backend-backed operations	13
Figure 5.1	Landing page and dashboard	18
Figure 5.2	Résumé/profile page	18
Figure 5.3	Application management	19
Figure 5.4	Career Intelligence panel	19
Figure 5.5	Interview tracker/calendar	20
Figure 5.6	Additional implemented tools	20

LIST OF TABLES

Table	Title	Page
Table 3.1	Technology stack and role	9
Table 3.2	Software and system requirements	9
Table 4.1	Application-readiness scoring weights	12
Table 4.2	Backend API endpoint groups	13
Table 4.3	Documented feature implementation status	14
Table 4.4	Automated test suites and evidence status	14

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CORS	Cross-Origin Resource Sharing
DOCX	Office Open XML Word document
DOM	Document Object Model
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NLP	Natural Language Processing
PDF	Portable Document Format
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
VIT	Vellore Institute of Technology
Zod	TypeScript-first schema validation library
SQLite	Embedded relational database engine
CRUD API	API operations for create, read, update and delete
SPA	Single-Page Application

CHAPTER 1: INTRODUCTION

Job searching involves more than discovering vacancies. Candidates must prepare and adapt résumé content, record submissions, monitor application states, respond to recruiters, prepare for interviews, and remember follow-up dates. Managing these activities across multiple portals and separate documents can fragment information and make progress difficult to review. JobPilot is designed as a unified web workspace that brings application tracking together with candidate-profile processing, evidence-based skill assessment and career-preparation tools.

The project, titled JobPilot: AI-Powered Job Application Tracker and Career Intelligence Platform, combines a React and TypeScript frontend with a Python FastAPI backend. It provides application records, documents and reminders, as well as career-intelligence features such as résumé text extraction, job matching, a six-factor application-readiness score, Skill Proof (skill-evidence strength), a Career Twin Lite profile, next-best-action coaching, cover-letter drafting and interview preparation. AI requests are routed through the FastAPI backend to the configured Gemini API, and selected workflows have deterministic fallbacks. The application-management component is retained: JobPilot tracks applications, and users apply on the employer's external page.

The project follows a progression of Track → Match → Prove → Predict → Act → Improve. Track is application management; Match is candidate/job relevance; Prove is evidence-based skill assessment; Predict is the project-defined readiness assessment; Act is next-best-action recommendation; and Improve is iterative career development as the candidate updates their profile. These stages are decision-support aids driven by the user; none of them is autonomous, and none guarantees an interview or job offer.

1.1 Motivation

The motivation is to reduce the cognitive overhead of a multi-stage job search. Candidates often need to maintain a consistent view of their target roles, résumé evidence, application status and interview deadlines. A shared workspace can help make these activities easier to review and prioritize. The project also explores how a transparent, factor-based readiness score and evidence-oriented skill profile can provide actionable guidance instead of presenting a single unexplained match number.

1.2 Problem Statement

How can a web application combine job-application record keeping with résumé processing and career-preparation assistance, while maintaining user-scoped application data and providing graceful behavior when an external AI service is unavailable?

1.3 Objectives

- Provide a centralized interface for creating, updating, searching and reviewing job-application records.
- Support résumé text extraction and structured candidate-profile processing for supported file formats and text input.
- Provide candidate-to-job matching and a documented six-factor application-readiness assessment.
- Offer career-support workflows, including tailored cover-letter drafting, skill-evidence tracking and interview practice, where available in the implementation.
- Maintain application-related documents and reminders through backend API operations.
- Apply Firebase ID-token verification and user-scoped data access in backend routes.
- Document the implementation and its limitations using reproducible evidence rather than unsupported claims.

1.4 Scope

The scope covers the web interface, application-management services, résumé/profile processing, AI-assisted career-intelligence utilities, backend API, authentication integration, persistence model, configuration for deployment and automated test definitions present in the repository. The job catalogue is curated static data rather than a verified live job feed, the recruiter inbox is sample data, and quick-fill is clipboard assistance rather than external-site form-filling automation. Actual live deployment and the results of the latest test execution are not independently confirmed in this report.

CHAPTER 2: EXISTING WORK / LITERATURE REVIEW

JobPilot sits at the intersection of applicant tracking, information extraction, recommender systems and AI-assisted career preparation. These areas provide useful concepts for structuring candidate information and supporting job-search decisions. This review uses selected foundational and official technical references to explain design choices; it does not claim that the project has experimentally outperformed existing systems.

2.1 Information Extraction from Résumés

Résumé information extraction converts semi-structured documents into fields such as skills, education, experience and contact details. In JobPilot, client-side document processing and text sanitization are separated from the later interpretation stage. This separation makes the pipeline easier to reason about: extraction produces text, while profile parsing attempts to interpret it. PDF.js provides a browser-oriented PDF processing library [1]. Supported formats and actual parsing behavior must be understood from the implementation rather than assumed to cover every résumé layout.

2.2 Semantic Matching and Recommendation

Text matching methods can compare job descriptions with candidate résumé content using tokens, required-skill overlap or vector representations. Semantic similarity can identify related wording that literal keyword matching may miss, but scores depend on representation, preprocessing and the quality of the input. Research on neural information retrieval illustrates the role of learned representations in ranking text passages [2]. JobPilot's available code includes deterministic matching and AI/embedding-related paths; the report therefore distinguishes a heuristic match score from a validated recruitment outcome.

2.3 Explainable Decision Support

Decision-support tools are more useful when users can see the factors that contribute to a recommendation. JobPilot documents a weighted readiness formula with six components and provides positive signals and areas that reduce readiness. This design aligns with the broader principle that an explanation should help users understand the basis of a system's output [3]. The score should be interpreted as a project-defined readiness heuristic, not a statistically calibrated probability of receiving an offer unless outcome-based validation is later performed.

2.4 AI Assistants and Structured Outputs

Generative language models can assist with drafting and summarization, but their output may be incomplete, malformed or inconsistent. Schema validation and deterministic fallback handling reduce reliance on unvalidated responses. Zod provides schema-based runtime validation in TypeScript [4], while Gemini provides an API interface for connecting to supported language models [5]. These integrations do not guarantee correctness; the implementation uses validation and fallback logic to handle some failure cases.

2.5 Relevance to JobPilot

The reviewed concepts inform four implementation decisions: separate document extraction from interpretation; explain scoring through named factors; validate AI-derived structured data; and keep core application record management distinct from optional AI assistance. The present report describes these as design choices in JobPilot. A controlled user study, comparison against a labelled dataset, or validation against real hiring outcomes would be required for claims about recruitment effectiveness.

CHAPTER 3: FRONTEND, BACKEND AND SYSTEM REQUIREMENTS

JobPilot is organized as a browser-based frontend and a Python API backend. The frontend contains route-level pages and service modules. The backend exposes REST endpoints, applies Firebase ID-token verification, hosts the AI endpoints that call the Gemini API, and uses SQLAlchemy models backed by SQLite by default (configurable through DATABASE_URL). Deployment configuration files are included, but a running production deployment was not verified in this report.

3.1 Technology Stack

Layer / Technology	Role in JobPilot
React 19	Component-based user interface [11].
TypeScript	Static typing for frontend modules and service contracts.
Vite	Frontend development server and build tooling [10].
TanStack Router / Query	Route organization and asynchronous data management where used [9].
Tailwind CSS v4	Utility-based interface styling.
Zod	Runtime validation of frontend data and AI response structures [4].
Python FastAPI	Backend HTTP API and endpoint definitions [6].
Pydantic	Request/response schema validation.
SQLAlchemy + SQLite	Object-relational mapping and relational persistence [7].
Firebase Authentication / Admin SDK	Identity sign-in integration and server-side ID-token verification [8].
Gemini API	External language-model API called only by the FastAPI backend; GEMINI_API_KEY and GEMINI_MODEL are backend configuration [5].
Vercel / Render configuration	Deployment configuration for frontend and backend; live status unverified.

3.2 Software Requirements

Requirement	Description
Frontend runtime	Node.js version specified by the Frontend package configuration (Node.js $\geq$ 22.12.0 and npm $\geq$ 10.0.0).
Frontend dependencies	Install using the package manager and lockfile supplied in the Frontend directory.
Backend runtime	Python 3.9+ is stated in repository setup documentation; confirm compatibility with the installed environment [12].
Backend packages	FastAPI, Uvicorn, Pydantic, SQLAlchemy, Firebase Admin, python-multipart, pytest and httpx.
Database	SQLite local database by default; a DATABASE_URL override is supported by code.
AI configuration	GEMINI_API_KEY and GEMINI_MODEL are configured on the backend, not in the browser; deterministic fallback logic exists for selected tasks when Gemini is unavailable.
Authentication configuration	Firebase client configuration and backend service credentials are required for the authenticated path; X-User-Id is accepted only when DEMO_MODE is explicitly enabled for development/demonstration.
Browser	Modern browser with JavaScript support; voice assistance additionally depends on available browser speech-recognition APIs.

3.3 Hardware Requirements

The repository does not define benchmarked minimum hardware specifications. For development, a general-purpose computer capable of running a modern browser, Node.js, Python and a local SQLite database is sufficient for the software stack described. Actual production capacity depends on hosting configuration, concurrent users, document sizes, external AI latency and database workload; no load test was supplied to establish a numeric capacity.

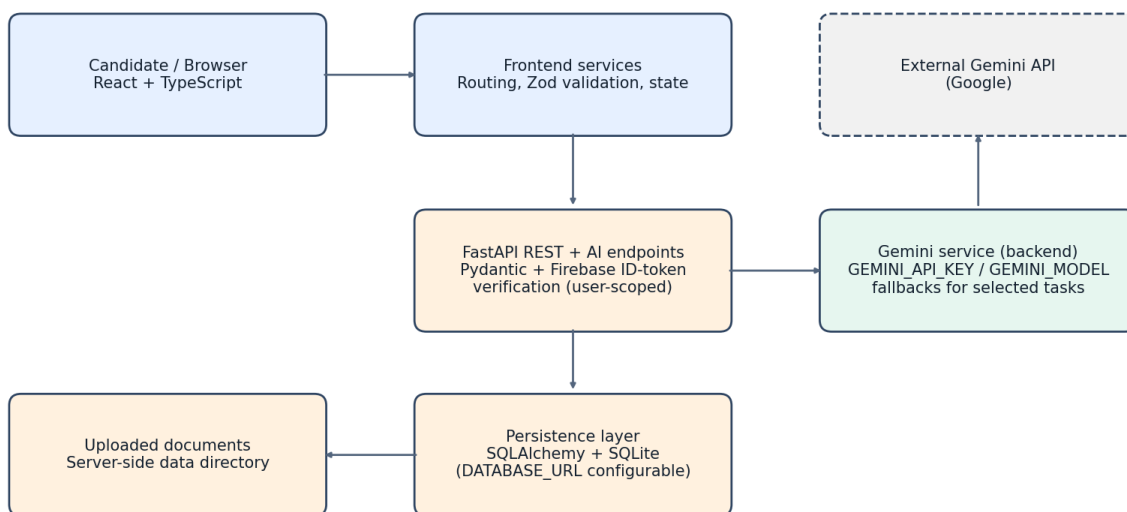
CHAPTER 4: METHODOLOGY

This chapter describes the architecture and implemented workflows identified from the repository and its Stage 1 analysis. Where functionality is only described by the README, or where execution was not verified, the distinction is stated explicitly. The diagrams are logical views derived from the code organization; they are not measured runtime traces.

4.1 System Design and Architecture

The application separates the presentation layer, frontend services, career-intelligence logic, backend API and persistence. The browser presents route-level pages and invokes service modules. Backend handlers apply an authentication dependency, validate input and scope database queries to the verified user identifier. AI requests are sent from the frontend to protected FastAPI AI endpoints, which call a Gemini service module and, through it, the Gemini API; provider credentials therefore remain outside the browser. SQLAlchemy maps application, document and reminder records to SQLite tables (or the database named by DATABASE_URL); uploaded files are stored in a configured server-side data directory with metadata in the database.

JobPilot: logical system architecture (repository-derived)



Provider credentials stay on the backend; the browser calls only protected FastAPI endpoints. Deployment files exist, but a live deployment was not verified.

Figure 4.1: JobPilot logical system architecture (derived from repository structure). AI requests pass through the FastAPI backend to the configured Gemini API.

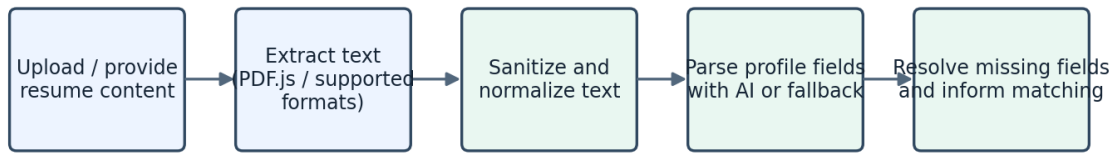
4.2 Frontend Design

The frontend is implemented with React and TypeScript and organized using route files. Major screens listed in the repository include the dashboard, job browsing, application list and application detail, recruiter inbox, profile, tracker and settings. The project uses reusable interface components for résumé/profile tools, career-intelligence panels, modals, calendar interactions and quick-fill operations. A route or component existing in source code does not itself establish that every path is connected to a live external data provider.

4.3 Résumé Ingestion and Profile Processing

The documented ingestion pipeline accepts supported résumé content and extracts text from supported formats. The frontend résumé parser uses PDF.js for PDF text extraction and has paths for additional text/document formats. Extracted content is sanitized to remove non-printable control characters and normalize text before profile interpretation. Structured extraction can use the backend AI endpoints (Gemini, when configured); when a call is unavailable or fails, selected flows use deterministic fallback behavior. Résumé parsing accuracy was not measured against an annotated benchmark in the supplied evidence.

Résumé ingestion and profile-processing pipeline



Extraction and interpretation are different stages. External LLM calls depend on configuration; local deterministic fallback behavior is available for supported operations.

Figure 4.2: Résumé ingestion and profile-processing pipeline.

4.4 AI Integration and Validation

AI requests follow the path: user/browser → React/TypeScript frontend → protected FastAPI AI endpoints → Gemini service module → Gemini API. The Gemini credentials (GEMINI_API_KEY and the model name GEMINI_MODEL) are backend configuration and are not exposed to the browser; the frontend never calls the provider directly. The AI endpoints sit behind the same authentication dependency as the other routes, so requests are associated with the verified user. The AI functionality covers résumé parsing, job matching, cover-letter generation, interview assistance and career-intelligence outputs.

Availability of live model responses depends on backend configuration, network access and provider availability. Where a fallback is implemented, a user may receive deterministic or rule-based output instead of a live model response; not every workflow has such a fallback. Zod schemas validate selected runtime data shapes in the frontend, and Pydantic validates backend request/response structures. AI output is not guaranteed to be correct or complete and should be reviewed by the user before it is used in application materials.

4.5 Job Catalogue and Matching

The job catalogue is a curated static collection, not a verified live integration with a large external job database. The interface supports role browsing, filters and links to employers' application pages, but the claim of "50,000+ live jobs" is not supported by the catalogue. Listing URLs and their verification are being improved separately, and this report does not claim that every catalogue entry is live or verified. A deterministic matching function uses available résumé text, candidate skills and job requirements, and other AI/embedding-assisted paths may be available when configured. Matching scores are technical relevance estimates based on implemented logic and inputs, not proof of job eligibility or a predicted hiring result.

4.6 Application-Readiness Scoring

The career-intelligence module implements a weighted readiness formula with six named factors. The weights sum to 100 percent. The resulting score is categorized into readiness bands and is accompanied by positive signals and areas that may reduce readiness. This provides an interpretable breakdown, but the repository evidence does not establish that the values are calibrated against historical hiring outcomes. A reader should use the score as guidance within the application, not as an objective probability of success.

Six-factor application-readiness workflow

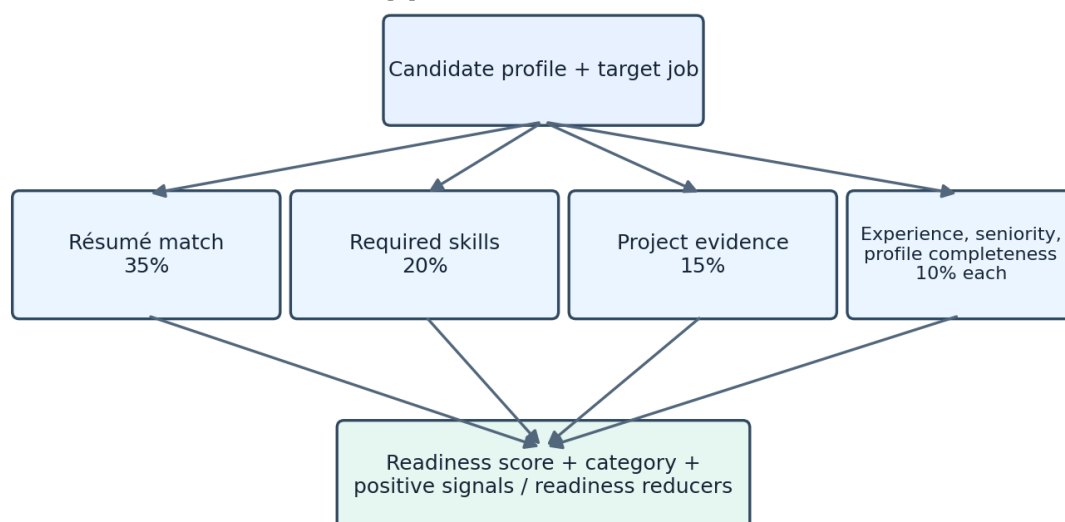


Figure 4.3: Six-factor application-readiness workflow.

Factor	Weight	Interpretation
Résumé match	35%	Relevance between the résumé/profile and the target job.
Required skills	20%	Overlap with skills specified for the role.
Project evidence	15%	Evidence of related practical projects.
Experience relevance	10%	Relevance of the candidate's experience.
Seniority fit	10%	Alignment between candidate profile and target seniority.
Profile completeness	10%	Completeness of available candidate information.
Total	100%	Weighted readiness heuristic.

Table 4.1: Documented application-readiness scoring weights.

4.7 Skill Evidence and Career-Twin Logic

The career-intelligence module represents skill evidence through levels such as claimed, résumé-mentioned, demonstrated by project and verified through a technical challenge. The career-twin representation also records skills, projects, experience years, strengths, weaknesses, likely interview areas and selected performance notes. These structures support personalized guidance. Their value depends on the quality of profile evidence entered or extracted; an unverified claim should not be interpreted as proof of competence.

4.8 Next-Best-Action Workflow

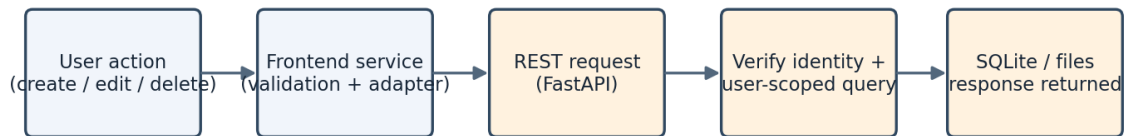
The documented workflow connects candidate/job context to readiness assessment, skill evidence, career-twin state and a prioritized next-best action. Suggested actions may include improving a skill, verifying skill evidence, adding a project, updating résumé content, completing the profile, preparing for an interview or following up. Where an action is completed, the system can update profile state and recompute career-intelligence outputs. The score impact is an application-generated estimate, not a guarantee of improved hiring outcomes.

4.9 Application Records, Documents and Reminders

The FastAPI backend defines ApplicationDB, DocumentDB and ReminderDB models. Application endpoints support listing, fetching, creating, updating and deleting application records, with optional filtering by status/source and company or role search. Document routes support metadata listing, upload, download and deletion. Reminder routes support listing, creation, update and deletion. Dashboard endpoints aggregate counts by status, recent applications and upcoming

follow-ups. The backend filters queries using the user identifier returned by its authentication dependency.

Application-data lifecycle (backend-backed operations)



Some frontend services also support local/demo persistence. The chosen mode and authentication configuration affect the runtime path; not every feature uses the same persistence mechanism.

Figure 4.4: Application-data lifecycle for backend-backed operations.

4.10 API and Data Model

The API uses FastAPI for routing and Pydantic for request/response schemas. SQLAlchemy maps relational records and defines relationships from applications to associated documents and reminders. The default database is SQLite and its location is configurable through the DATABASE_URL environment setting. Document files are written to a configured server-side data directory and database records store metadata and the physical storage reference; Firebase Storage is not used for documents. The upload handler enforces a 5 MB size limit, checks the request MIME type against an allow-list and validates the file content/signature for the supported PDF, DOC and DOCX formats; document access is scoped to the authenticated user. These checks reduce the risk of mislabelled uploads but do not constitute antivirus or malware scanning, and the upload path has not been security-tested.

API group	Representative operations	Purpose
Health	GET / ; GET /api/health	Service status response.
Applications	GET/POST /api/applications	List and create user-scoped application records.
Application detail	GET/PATCH/DELETE /api/applications/{app_id}	Retrieve, edit or delete one application.
Documents	GET /api/documents ; POST /api/documents/upload	List and upload documents.
Document download	GET /api/documents/{doc_id}/download	Download an authorized stored document.
Document deletion	DELETE /api/documents/{doc_id}	Remove document metadata and attempt to remove the stored file.
Reminders	GET/POST /api/reminders	List and create reminders associated with applications.
Reminder detail	PATCH/DELETE /api/reminders/{rem_id}	Update or delete a reminder.
Dashboard stats	Dashboard statistics endpoint(s)	Return totals, status counts and upcoming follow-ups.

Table 4.2: Backend API endpoint groups. Refer to the FastAPI source for the complete endpoint inventory.

4.11 Authentication and Security Considerations

The backend authentication module verifies Firebase ID tokens using the Firebase Admin SDK and derives the user identifier from the decoded token. An X-User-Id development bypass is accepted only when DEMO_MODE is explicitly enabled and is intended solely for development and demonstration. Routes use the verified user identifier in database filters, so application, document and reminder records are user-scoped, and AI endpoints are likewise reached through the backend. Provider credentials (Gemini key, Firebase service credentials) are held in backend

configuration. Document uploads are subject to size, MIME-type and content/signature checks as described in Section 4.10, and frontend URL validation restricts supported schemes in application URLs. These are implementation safeguards, not a comprehensive security audit: formal threat modelling, rate limiting review, antivirus scanning of uploads and production penetration testing were not evidenced and remain future validation work.

4.12 Feature Status: Implemented and Limitations

Feature	Evidence-based status	Reporting note
Résumé extraction and profile processing	Implemented in source	Supported formats and parsing behavior should be described without claiming benchmarked accuracy.
Six-factor readiness scoring	Implemented in career-intelligence module	Formula documented; not a calibrated hiring probability.
AI résumé parsing, cover-letter drafting, interview assistance and career intelligence	Implemented with configured AI calls and selected deterministic fallbacks	Live LLM use depends on backend Gemini configuration and provider availability.
Skill-evidence tracking	Implemented in career-intelligence module	Evidence depends on profile data and verification steps.
Application CRUD, documents, reminders and authentication	Implemented in FastAPI source	Runtime configuration and production deployment still require environment setup.
Job listings presented as 50,000+ live jobs	Not supported by inspected catalogue	Repository contains a curated static job list rather than verified live portal integration.
Recruiter inbox	Sample data only	No live mailbox or recruiter API integration verified.
Voice navigation	Browser speech input + keyword route matching	Not a custom NLP navigation model.
Quick-fill	Clipboard-based field-copy helper	Not verified as automatic external website form filling.
Automatic portal application submission	Not implemented in inspected repository	Links out to application portals instead.
Production deployment	Configuration files present	Live deployment was not verified.

Table 4.3: Evidence-based implementation status and limitations.

4.13 Testing Strategy and Available Evidence

The repository contains automated frontend (Vitest) and backend (pytest) test suites covering application logic, career intelligence and API behavior. The earlier source-level inventory (78 frontend test blocks and six backend test functions) predates later repository changes and should not be treated as the current count. The results of an executed run of the current suites, including build and type-check status, are not recorded in this report; the final submission should report the results of the latest executed test run together with the exact commands used.

Layer	Source-level inventory	Evidence status
Frontend	Vitest suites (application logic and career intelligence); count to be taken from the latest run	Suite present; execution result for the current repository state to be reported from the latest run.
Backend	pytest suite (API behavior); count to be taken from the latest run	Suite present; execution result for the current repository state to be reported from the latest run.

Table 4.4: Automated test suites and evidence status.

A complete verification run should install the documented dependencies, configure a test environment, execute the frontend and backend suites together with the frontend type-check and production build, record the exact commands and outputs, and resolve any environment-dependent failures. Manual testing should also cover authentication failure, data isolation, invalid upload types and mismatched file signatures, size limits, application CRUD operations, reminder flows and the local fallback when no AI key is supplied.

4.14 Results and Discussion

The repository inspection confirms the existence of the main frontend and backend modules and shows that several career-intelligence and AI-assisted workflows are implemented in source code. It also identifies a gap between some README language and the inspected behavior: job listings are static curated records, the inbox uses sample threads, and quick-fill copies profile fields rather than performing automatic external-site submission. These boundaries should be communicated transparently in a project demonstration.

No measured user study, load-test benchmark, labeled résumé extraction evaluation, verified live job feed, or hiring-outcome dataset was supplied. Therefore, claims about user productivity gains, matching accuracy, application success rate, latency or improved hiring probability cannot be quantified in this report. The results are primarily implementation-level findings. The next useful evaluation steps are functional testing, security review, extraction-quality measurement and usability feedback from representative users.

4.15 Individual Contributions

Member	Individual Contribution	Brief
Harshit Arora	Frontend Development & UI Integration	Designed and developed the React/Vite frontend, including dashboard, job browsing, application tracker, profile/resume interfaces, application forms, Career Intelligence UI, navigation, responsive layouts, and integration of frontend components with backend APIs.
Ishaan Verma	Backend & API Development	Developed the FastAPI backend, REST API endpoints, request/response handling, authentication integration, application/job/profile APIs, database communication, error handling, and frontend-backend integration.
Aashish Pandey	AI & Career Intelligence Module	Developed the AI-powered career intelligence functionality including resume/job matching, Application Success Prediction, Skill Proof analysis, Career Twin, interview preparation/simulation, AI recommendations, and integration of AI services with the application
Udit Narayan Rout	Database, Authentication & Data Management	Worked on database architecture and data persistence, including user profiles, jobs, applications, documents and related entities. Implemented/managed authentication flow, data validation, CRUD operations, and reliable storage/retrieval of application data.

Kartikey Tiwari	Testing, Deployment & System Integration	Handled system testing, API testing, integration testing, debugging, deployment configuration and production validation. Worked with Vercel/Render deployment, environment configuration, health checks, error handling and verification of end-to-end application workflows.
-----------------	--	---

CHAPTER 5: CONCLUSION

5.1 Conclusion

JobPilot is a full-stack AI-powered career intelligence platform that combines job-application management, candidate profiling, job matching, application-readiness assessment and AI-assisted career preparation. The implementation combines a React and TypeScript frontend with a FastAPI backend, SQLAlchemy data models and SQLite persistence by default. Firebase ID-token verification is integrated into backend authentication and database operations are user-scoped, while AI requests are routed through the backend to the Gemini API so that provider credentials stay outside the browser; selected workflows have deterministic fallbacks.

The main contribution is the connected workflow rather than any single feature:

Candidate Profile → Job Matching → Application Success Predictor → Skill Proof → Career Twin → Next Best Action → Interview Preparation

The candidate profile feeds job matching; the six-factor readiness assessment (a project-defined heuristic) explains where a profile is strong or weak for a role; Skill Proof grades the strength of evidence behind each skill; the Career Twin Lite keeps a structured view of the candidate; the next-best-action coach turns these signals into a prioritized task; and interview preparation addresses likely interview areas. This supports the Track → Match → Prove → Predict → Act → Improve progression. The system supports the candidate's decisions; it does not predict hiring outcomes and does not guarantee employment.

The inspection also establishes limits that should be acknowledged in the final demonstration: the curated job catalogue is not a live feed, the recruiter inbox is sample data, quick-fill is clipboard assistance rather than external form automation, and applications are not submitted automatically (users are redirected to the employer's page). Deployment configuration exists, but live deployment was not verified, and the results of the latest test run should be reported separately at final submission.

5.2 Limitations

- The available job catalogue is static curated data; no verified live external job feed was found.
- The recruiter inbox uses sample threads and is not connected to a live email or recruiting system.
- The quick-fill helper copies candidate fields to the clipboard; automated cross-site form filling and application submission are not implemented.
- AI-based features depend on configuration, network access and provider availability; selected fallbacks may return rule-based content rather than model-generated analysis.
- No benchmark of résumé extraction quality, matching quality, user productivity or hiring outcomes was provided; there is no large labelled résumé dataset or hiring-outcome dataset, and the readiness score is not a statistically calibrated probability of an interview or offer.

- Test suites exist in the repository, but this report does not record the results of the latest execution, so passing status is not established here.
- Production readiness depends on correctly configured secrets, hosting, persistent storage and further security and operational validation; uploads are validated by size, MIME type and file signature but are not antivirus-scanned, and no penetration test or formal threat model has been performed.

5.3 Future Enhancements

The following enhancements are planned future work for the next two project reviews. None of them is part of the currently implemented functionality.

Phase 3 — Real-Time Career Intelligence

- **Real-time job intelligence:** authorized job-provider APIs, continuously updated listings, listing verification, expiration detection, deduplication and job-metadata freshness.
- **Job-specific résumé optimization:** identify missing requirements, recommend résumé changes and generate role-specific résumé versions while preserving factuality.
- **Evidence-based skill graph:** connect skills with résumé evidence, projects, certifications, internship/experience evidence and technical verification, and improve the Skill Proof visualization.
- **Advanced interview intelligence:** role-specific technical and behavioral questions, AI interview simulation, answer evaluation, weak-area identification and improvement tracking.
- **Enhanced Career Twin:** continuously update the candidate state from skills, projects, experience and application activity, identify likely interview areas and improve personalized recommendations.

Phase 4 — AI Career Agent

- **Personalized career goals:** user-defined career goals that guide recommendations.
- **Continuous next-best-action recommendations:** continuously refreshed next-best-action recommendations.
- **Application outcome analysis:** learn from application and interview outcomes, identify patterns and improve recommendations; this requires real outcome data and would be the basis for any future calibration of the readiness score.
- **Candidate-specific opportunity ranking:** rank opportunities for the specific candidate.
- **Career-market intelligence:** skill demand, role trends, technology trends and salary/market information where reliable data is available.
- **Continuous feedback loop:** use application outcomes, interview outcomes, skill development and profile changes to improve future recommendations.

Supporting work carried forward

- **Integrations:** recruiter communication through a supported mail provider with explicit authorization and privacy safeguards; any browser-extension form assistance must require user confirmation before submission.
- **Validation:** evaluate résumé parsing against annotated examples, add automated CI with recorded test reports, access-control tests and a production security review, and conduct usability and accessibility studies.

Long-term, JobPilot aims to evolve from an application tracker into an AI-powered career operating system that continuously understands the candidate, analyzes the job market, identifies skill gaps, evaluates opportunity fit and recommends the highest-impact actions for career progression.

5.4 Project Screenshots

The Career Intelligence panel (Figure 5.4) is the project's central differentiating feature. The captured view shows résumé match, application readiness, Career Twin fit and a next-best-action card for a selected role; the figures shown are outputs of the implemented heuristics for sample profile data, not measured hiring probabilities.

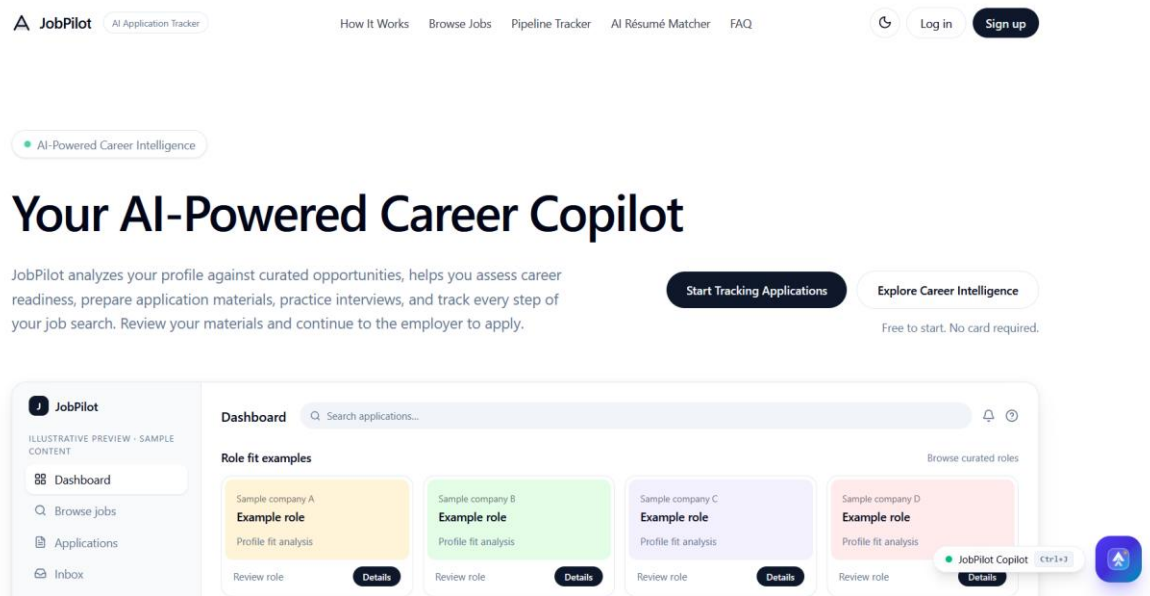


Figure 5.1: Landing page and dashboard

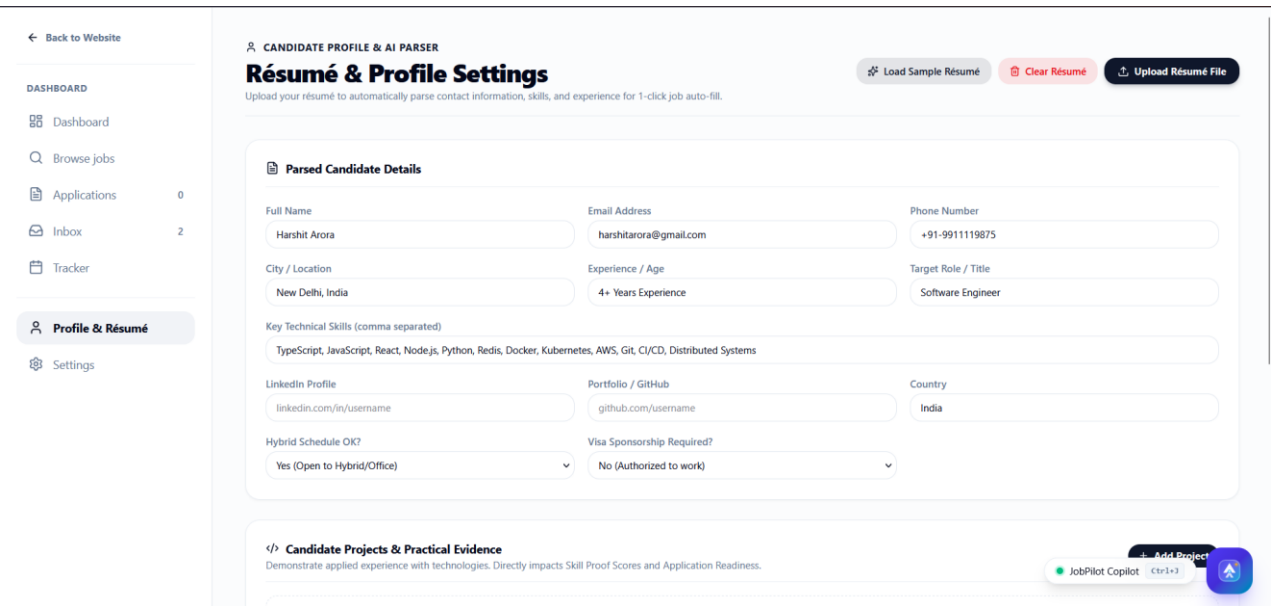


Figure 5.2: Résumé/profile page

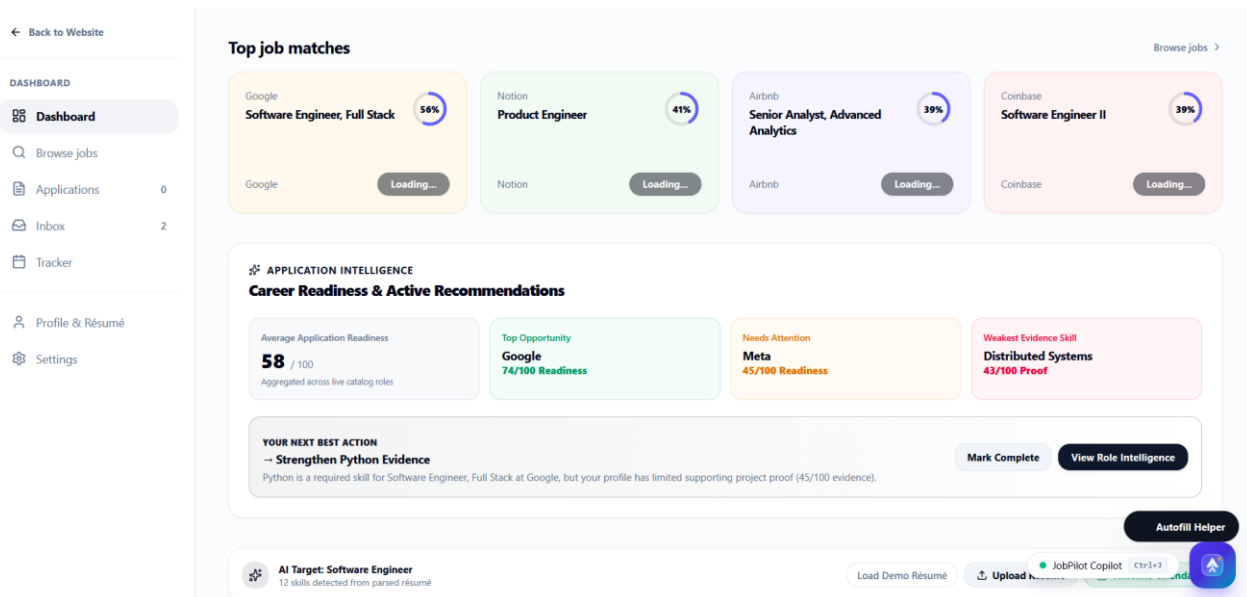


Figure 5.3: Application management

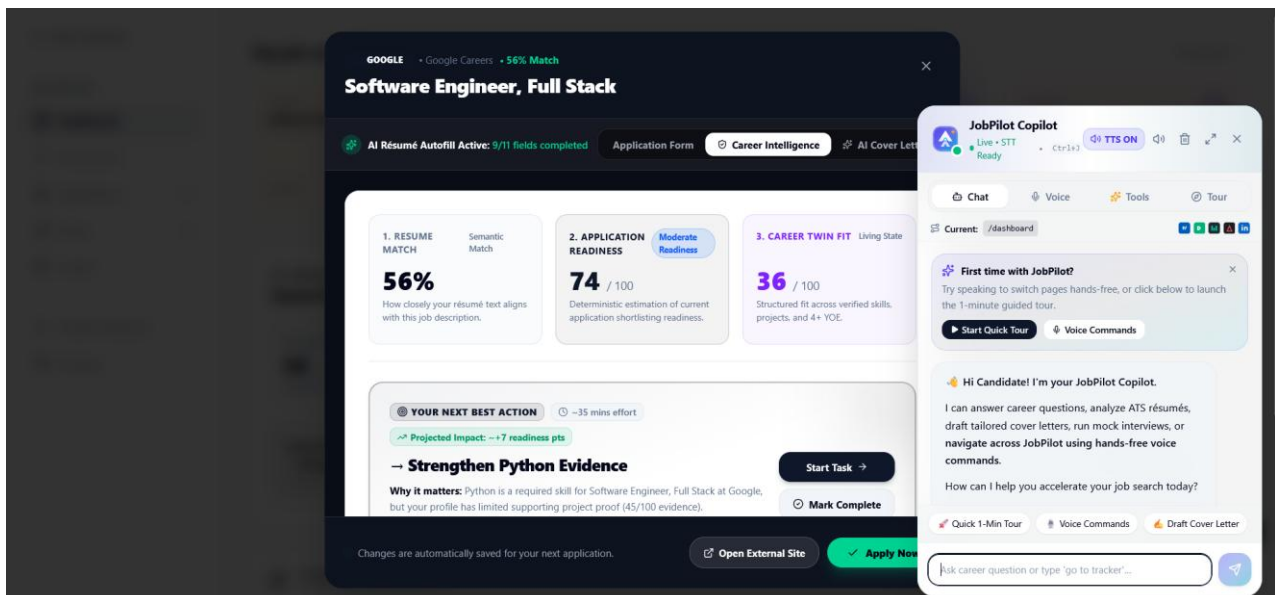


Figure 5.4: Career Intelligence panel (central feature: résumé match, application readiness, Career Twin fit and next best action)

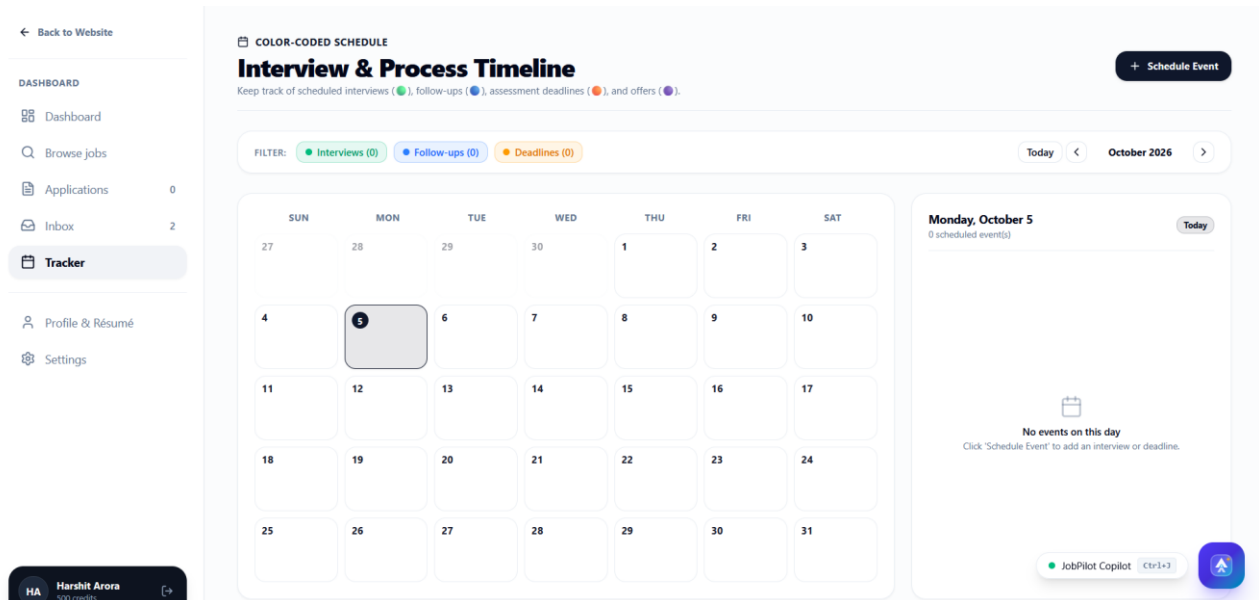


Figure 5.5: Interview tracker/calendar

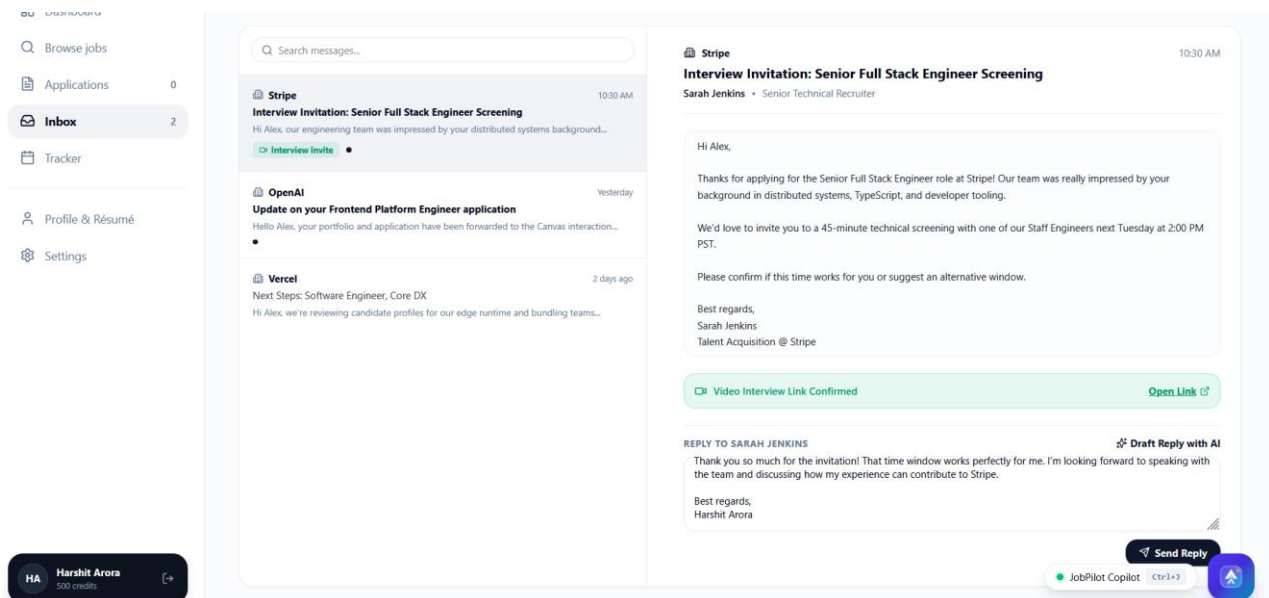


Figure 5.6: Additional implemented tools

5.5 Final Outcome

The outcome is a full-stack AI-powered career intelligence platform, documented from its source, that supports job-application management and a connected set of career-preparation workflows. Its current scope and limitations are recorded in this report so that real-time job data, deeper evidence-based skill analysis and the Phase 3 and Phase 4 enhancements can be evaluated against an accurate baseline.

REFERENCES AND PUBLICATIONS

- [1] Mozilla. “PDF.js: A general-purpose, web standards-based platform for parsing and rendering PDFs.” GitHub repository and project documentation. <https://github.com/mozilla/pdf.js>
- [2] V. Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering.” Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- [3] M. T. Ribeiro, S. Singh, and C. Guestrin. “Why Should I Trust You? Explaining the Predictions of Any Classifier.” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 1135–1144. <https://doi.org/10.1145/2939672.2939778>
- [4] Colinhacks. “Zod: TypeScript-first schema validation with static type inference.” Project documentation. <https://zod.dev/>
- [5] Google. “Gemini API documentation.” Google AI for Developers. <https://ai.google.dev/gemini-api/docs>
- [6] FastAPI. “FastAPI Documentation.” <https://fastapi.tiangolo.com/>
- [7] SQLAlchemy. “SQLAlchemy Documentation.” <https://docs.sqlalchemy.org/>
- [8] Firebase. “Verify ID Tokens.” Firebase Admin SDK documentation. <https://firebase.google.com/docs/auth/admin/verify-id-tokens>
- [9] TanStack. “TanStack Router Documentation.” <https://tanstack.com/router/latest>
- [10] Vite. “Vite Documentation.” <https://vite.dev/guide/>
- [11] React. “React Documentation.” <https://react.dev/>
- [12] Python Software Foundation. “Python Documentation.” <https://docs.python.org/3/>